

Unidade I

1 FUNDAMENTOS DE C#

C# é uma linguagem de programação criada pela empresa Microsoft. Ela serve para construir muitos tipos de software: sites, serviços em nuvem, programas de computador, aplicativos para celular e jogos. É moderna, com checagem de tipos (ajuda a evitar erros) e gerenciamento automático de memória, além de ter recursos de produtividade como `async/await` e LINQ. C# é a linguagem principal da plataforma .NET.

.NET é a plataforma que executa programas escritos em C#. A edição .NET 10 é o lançamento de 2025 (ciclo anual da Microsoft) e é um LTS (Long Term Support)

1.1 Sintaxe, tipos, nullability, var, organização de solução/projeto

A linguagem C# pertence à família das linguagens derivadas de C, possuindo uma sintaxe bastante familiar para quem já utilizou C, C++, Java ou JavaScript. Nela cada instrução termina com ponto e vírgula (;) e blocos de código são delimitados por chaves { e }. Os identificadores (nomes de variáveis, métodos, classes etc.) são case-sensitive, ou seja, diferencia maiúsculas de minúsculas, inclui as estruturas de controle usuais, como `if/else` para condicionais, `switch` para seleção de casos e laços de repetição `for`, `while` e `foreach`, com sintaxes muito semelhantes às do C/C++.

No código tradicional, começamos declarando um namespace (Exemplo) para organizar logicamente a classe Program. Dentro da classe, definimos o método Main com o modificador `static`, indicando que é um método de classe (e não de instância). O método Main sem parâmetros é convencionalmente o ponto de entrada de programas C# executáveis – ou seja, é a primeira função a ser invocada quando o programa é iniciado. Nele, fazemos a chamada `Console.WriteLine` para produzir saída no console.

C# é uma linguagem fortemente tipada, o que significa que cada variável e constante no código possui um tipo bem definido em tempo de compilação. Toda expressão em C# avalia para um tipo específico, conhecido pelo compilador, e operações envolvendo valores geralmente só são permitidas se os operandos forem de tipos compatíveis. Esse forte sistema de tipos promove a detecção precoce de erros: o compilador (ou mesmo as ferramentas do ambiente de desenvolvimento) emite erros ou avisos caso tentemos usar uma variável de modo inconsistente com seu tipo declarado.

Algumas dessas palavras-chaves são `int` (em C#), que corresponde ao tipo inteiro de 32 bits assinado (`System.Int32` na plataforma .NET); da mesma forma, `double` é um tipo de ponto flutuante de 64 bits (`System.Double`); `char` representa um caractere Unicode de 16 bits (`System.Char`); `bool` é um valor booleano lógico (`System.Boolean`, que pode ser `true` ou `false`), e assim por diante. O quadro 1 ilustra alguns dos tipos primitivos e seus propósitos.

Tipos por valor são aqueles que armazenam diretamente seus dados. Isso significa que uma variável de um tipo por valor contém em si o valor atribuído, e não uma referência a um valor em outro lugar. Os tipos numéricos primitivos (int, double etc.), bool, char e também os tipos definidos com struct (estruturas) são tipos por valor. Por padrão, instâncias de tipos por valor são alocadas na stack (pilha) de execução ou embutidas dentro de estruturas de dados maiores. Uma consequência importante é que, ao atribuir uma variável de tipo por valor a outra, realiza-se uma cópia dos dados.

Tipos por referência, por outro lado, armazenam referências (endereços) para objetos alocados na heap (área de memória dinâmica). Class (classes), arrays (vetores) e string (cadeia de caracteres) são exemplos de tipos por referência. Quando declaramos uma variável de um tipo por referência, inicialmente ela não aponta para nenhum objeto concreto e seu valor padrão é null (nulo), indicando nenhuma referência válida. Para obter um objeto real, geralmente usamos o operador new para instanciar a classe, o que aloca memória na heap e retorna uma referência para essa região de memória. A variável então armazena esse ponteiro (de forma segura, sem aritmética de ponteiros explícita na linguagem gerenciada). Ao atribuir uma variável de referência a outra, não copiamos os dados do objeto em si, mas sim a referência (o endereço).

Obs: Instanciar a classe significa criar um objeto concreto a partir de uma classe

Em memória de programas, stack e heap descrevem estratégias distintas de organizar dados ao longo da execução. A primeira se assemelha a uma pilha de pratos: tudo que entra por cima sai por cima, em ordem estritamente last-in, first-out (LIFO). A segunda lembra um almoxarifado: há espaço para itens de tamanhos variados, guardados e recuperados conforme a necessidade, sem seguir a mesma rigidez de ordem.

Quando a instrução envolve new, como em var pessoa = new Pessoa();, a instância resultante é criada no heap gerenciado, enquanto a variável local retém apenas uma referência a esse objeto.

parei na pag 18